Système d'exploitation TD7 : pipe, dup2

mars 2026 — Pierre Rousselin

Exercice 1 : pipe et dup2

On considère le squelette de programme suivant :

```
void die(const char *msg, int status);
void enfant(void);
void parent(void);
int tube[2];
int main(void)
{
    if (pipe(tube) < 0)
        die("pipe", 1);
    switch (fork()) {
    case -1: die("fork", 1);
    case 0:
         enfant();
         return 0;
    }
    parent();
    int stat;
    wait(&stat);
    if (WTERMSIG(stat))
        fprintf(stderr, "enfant tué par signal %d\n", WTERMSIG(stat));
    return 0;
}
void die(const char *msg, int status)
{
    perror(msg);
    exit(status);
}
```

Dans chacun des cas suivants, donner le comportement du programme, ou, le cas échéant, dire si c'est un comportement indéfini et pourquoi.

1.
```
void enfant(void)
{
    close(tube[0]);
    write(tube[1], "foo\n", 4);
    write(tube[1], "bar\n", 4);
    close(tube[1]);
}

void parent(void)
{
    close(tube[1]);
    char buff[1024];
    ssize_t n;
    while ((n = read(tube[0],
          buff, 1024)) > 0) {
        write(1, buff, n);
    }
    close(tube[0]);
}
```

2.
```
void enfant(void)
{
    close(tube[0]);
    write(tube[1], "foo\n", 4);
    write(tube[1], "bar\n", 4);
    close(tube[1]);
}
void parent(void)
{
    close(tube[1]);
    char buff[1024];
    ssize_t n = read(tube[0], buff, 1024);
    write(1, buff, n);
    close(tube[0]);
}
```

3.
```c
void enfant(void)
{
    close(tube[0]);
    close(tube[1]);
    write(tube[1], "foo\n", 4);
    write(tube[1], "bar\n", 4);
}

void parent(void)
{
    close(tube[1]);
    char buff[1024];
    ssize_t n =
      read(tube[0], buff, 1024);
    write(1, buff, n);
    close(tube[0]);
}
```

4.
```c
void enfant(void)
{
    close(tube[0]);
    write(tube[1], "foo\n", 4);
    write(tube[1], "bar\n", 4);
    close(tube[1]);
}

void parent(void)
{
    close(tube[0]);
    char buff[1024];
    ssize_t n =
      read(tube[0], buff, 1024);
    write(1, buff, n);
    close(tube[0]);
}
```

5.
```c
void enfant(void)
{
    close(tube[0]);
    dup2(tube[1], 1);
    close(tube[1]);
    /* rendre les opérations
     sur stdout line-buffered */
    setvbuf(stdout, NULL, _IOLBF, 0);
    printf("foo\n");
    printf("bar\n");
    close(1);
}

void parent(void)
{
    close(tube[1]);
    dup2(tube[0], 0);
    close(tube[0]);
    char buff[1024] = { 0 };
    while (scanf("%1023s", buff) > 0) {
        printf("%s\n", buff);
    }
    close(0);
}
```

6.
```c
void enfant(void)
{
    close(tube[0]);
    dup2(tube[1], 1);
    /* rendre les opérations
     sur stdout line-buffered */
    setvbuf(stdout, NULL, _IOLBF, 0);
    printf("foo\n");
    printf("bar\n");
    close(1);
    sleep(100);
}

void parent(void)
{
    close(tube[1]);
    dup2(tube[0], 0);
    close(tube[0]);
    char buff[1024] = { 0 };
    while (scanf("%1023s", buff) > 0) {
        printf("%s\n", buff);
    }
    printf("lol\n");
    close(0);
}
```

--- * ---

Exercice 2 : Un tube en C

Écrire un programme qui affiche l'environnement du processus courant en triant la sortie avec le programme `sort`, exécuté par un processus enfant. Le processus parent attendra la fin du processus enfant avant de mourir. Rappel : l'environnement d'un processus est accessible avec la variable

```c
extern char **environ;
```

--- * ---